

SETUL DE INSTRUCȚIUNI AL μ P Z - 80 (IV)

1. Obiectul lucrării

Se finalizează studiul setului de instrucțiuni al microprocesorului Z-80 cu următoarele grupe:

- instrucțiunile de rotație și deplasări;
 - instrucțiunile de lucru cu circuitele de intrare-ieșire (I/O);
- Se prezintă aplicații cu acestea.

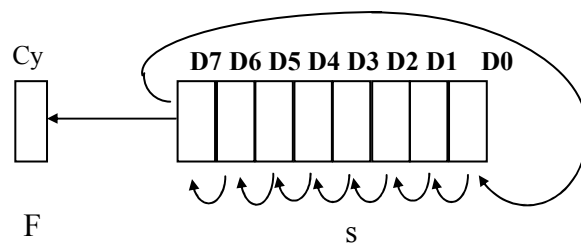
2. Breviar teoretic

XIV. Grupa instrucțiunilor de rotație și deplasare

A . Subgrupa instrucțiunilor de rotație

a). **RLC s** - (rotate left with carry) - rotație la stânga cu carry

Simbolic:



$s \in \{ r, (HL), (IX+e), (IY+e) \}$

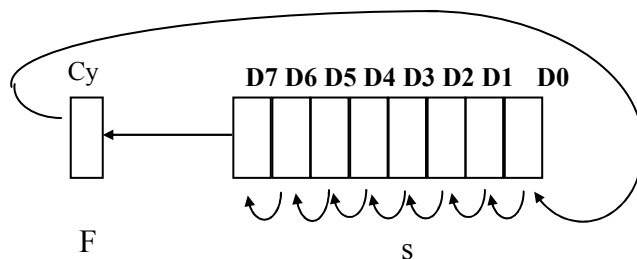
Apar situațiile:

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
RLC r	*	*	0	*	0	*	8	2	CBh 0 0 0 0 0 s s s	inițial A $\leftarrow$ 20h RLC A
RLC (HL)	*	*	0	*	0	*	15	2	CBh, 06h	final A $\leftarrow$ 40h
RLC (IX + e)	*	*	0	*	0	*	23	4	DDh, CBh e, 06h	daca HL $\leftarrow$ 100h inițial (100h) = 30h RLC (HL)
RLC (IY + e)	*	*	0	*	0	*	23	4	FDh, CBh e, 06h	final (100h) = 60h

Îndrumar de laborator 5

b). **RL s** - (rotate left through carry) - rotație la stânga prin intermediul bitului carry

Simbolic:



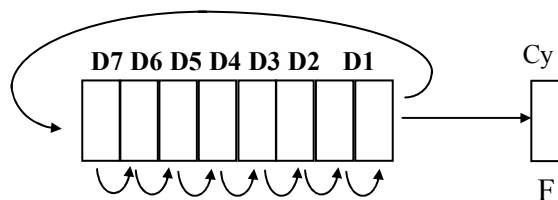
$s \in \{r, (HL), (IX+e), (IY+e)\}$

Apar situațiile:

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
RL A	-	-	0	-	0	*	4	2	CBh 0 0 0 1 0 s s s	inițial A ← 20h RL A
RL r	*	*	0	*	0	*	8	2	CBh 0 0 0 1 0 s s s	final A ← 40h
RL (HL)	*	*	0	*	0	*	15	2	CBh, 16h	dacă Cy = 0 sau
RL (IX + e)	*	*	0	*	0	*	23	4	DDh, CBh e, 16h	A ← 41h dacă
RL (IY + e)	*	*	0	*	0	*	23	4	FDh, CBh e, 16h	Cy = 1

c). **RRC s** - (rotate right with carry) - rotație la dreapta cu carry

Simbolic:



$s \in \{r, (HL), (IX+e), (IY+e)\}$

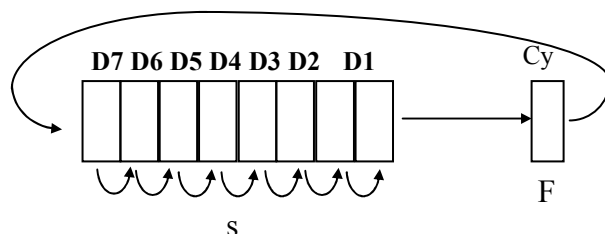
Apar situațiile:

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
RRC A	-	-	0	-	0	*	4	2	CBh 0 0 0 0 1 s s s	Dacă
RRC r	*	*	0	*	0	*	8	2	CBh 0 0 0 0 1 s s s	IX ← 100h e = 0
RRC (HL)	*	*	0	*	0	*	15	2	CBh, 0Eh	inițial
RRC (IX + e)	*	*	0	*	0	*	23	4	DDh, CBh e, 0Eh	(100h) = 40h
RRC (IY + e)	*	*	0	*	0	*	23	4	FDh, CBh e, 0Eh	RRC (IX+e)
										final (100h) =
										20h
										inițial A ← 80h
										RRC A
										final A ← 40h

Îndrumar de laborator 5

d). **RR s** - (rotate right through carry) - rotație la dreapta prin intermediul bitului carry

Simbolic:



$s \in \{r, (HL), (IX+e), (IY+e)\}$

Apar situațiile :

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
RR A	-	-	0	-	0	*	4	2	CBh 0 0 0 1 1 s s s	dacă $IX \leftarrow 100h$ $e=0$
RR r	*	*	0	*	0	*	8	2	CBh 0 0 0 1 1 s s s	inițial (100h)=40h RR (IX+e)
RR (HL)	*	*	0	*	0	*	15	2	CBh, 1Eh	final (100h) = 20h
RR (IX + e)	*	*	0	*	0	*	23	4	DDh, CBh e, 1Eh	dacă $Cy=0$ sau (100h)=21h dacă $Cy=1$
RR (IY + e)	*	*	0	*	0	*	23	4	FDh, CBh e, 1Eh	

OBSERVAȚIE . La microprocesorul Z-80, pentru acumulator există în plus 4 instrucțiuni de rotație, dar care sunt compatibile cu cele descrise anterior. Diferența între cele două tipuri de instrucțiuni apare doar din punctul de vedere al mnemonicii folosite și al codului mașină propriu-zis. Aceste instrucțiuni au apărut ca urmare a echivalenței software dintre microprocesoarele Z-80 și 8080. La 8080 existau instrucțiuni de rotație doar pentru acumulator (codul mașină având 1 octet).

Există următoarele echivalențe:

RRC A : CBh 0Fh

≡

RRCA : 0Fh

RR A : CBh 0Eh

≡

RRA : 0Eh

RL A : CBh 17h

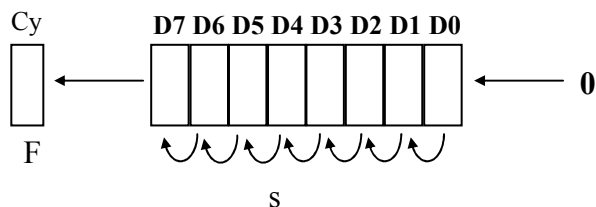
≡

RLA : 17h

RLC A : CBh 07h

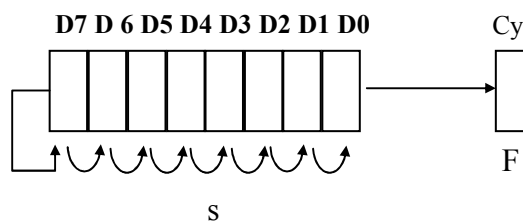
≡

RLCA : 07h

B . Subgrupa instrucțiunilor de deplasarea). **SLA s** - (shift left arithmetic) - deplasare stânga aritmetică
 $s \in \{ r, (HL), (IX+e), (IY+e) \}$

În primul bit D0 (bitul cms) se aplică “0” logic și are loc deplasarea cu o poziție spre stânga a celorlalți biți. Ultimul bit D7 (bitul cms), trece în indicatorul carry din registrul indicatorilor de condiții F.

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
SLA r	* * 0 * 0 *	8	2	CBh 0 0 1 0 0 s s s	dacă $X \leftarrow 100h$ e=0 inițial (100h)=80h
SLA (HL)	* * 0 * 0 *	15	2	CBh, 26h	SLA (IX+e) final (100h) = 0h iar Cy =1
SLA (IX + e)	* * 0 * 0 *	23	4	DDh, CBh e, 26h	
SLA (IY + e)	* * 0 * 0 *	23	4	FDh, CBh e, 26h	

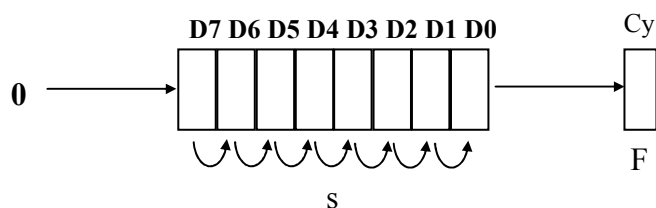
b). **SRA s** - (shift right arithmetic) - deplasare dreapta aritmetică
 $s \in \{ r, (HL), (IX+e), (IY+e) \}$

Bitul D7 (cms) se deplasează la dreapta și se repetă. Ceilalți biți se deplasează cu o poziție spre dreapta, iar ultimul bit D0 (cms) trece în indicatorul carry din registrul indicatorilor de condiții F.

Îndrumar de laborator 5

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
SRA r	* * 0 * 0 *	8	2	CBh 0 0 1 0 1 s s s	inițial B←81h SRA B final B=0C0h si Cy=1
SRA (HL)	* * 0 * 0 *	15	2	CBh, 2Eh	
SRA (IX + e)	* * 0 * 0 *	23	4	DDh, CBh e, 2h	
SRA (IY + e)	* * 0 * 0 *	23	4	FDh, CBh e, 2Eh	

c). **SRL** s - (shift right logic) - deplasare dreapta logică



$s \in \{ r, (HL), (IX+e), (IY+e) \}$

În bitul D7 (cms), se aplică “0” logic și are loc o deplasare spre dreapta cu o poziție a tuturor biților. Ultimul bit D0 (cmps) trece în indicatorul carry din registrul indicatorilor de condiții F.

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
SRL r	* * 0 * 0 *	8	2	CBh 0 0 1 1 1 s s s	inițial B←81h SRL B Final B←40h si Cy=1 daca IY←200h e=0 inițial (20h)=44h SRL (IY+e) final(20h)=22h Cy=0
SRL (HL)	* * 0 * 0 *	15	2	CBh, 3Eh	
SRL (IX + e)	* * 0 * 0 *	23	4	DDh, CBh e, 3Eh	
SRL (IY + e)	* * 0 * 0 *	23	4	FDh, CBh e, 3Eh	

OBSERVAȚIE. Instrucțiunile de deplasare, precum și cele de rotație pot fi folosite pentru efectuarea operațiilor aritmetice de înmulțire și împărțire. Să considerăm numărul binar 00000100₍₂₎, adică 4₍₁₀₎.

În cazul unei deplasări la stânga (situația unei instrucțiuni SLA), se obține numărul 00001000₍₂₎, adică 8₍₁₀₎. După încă o deplasare la stânga se obține 00010000₍₂₎, adică 16₍₁₀₎. Rezultă că o deplasare aritmetică stânga echivalează cu o înmulțire cu 2. În mod similar, o deplasare logică la dreapta echivalează cu o împărțire cu 2.

Îndrumar de laborator 5

Într-un alt exemplu, să considerăm operația de înmulțire a numerelor 4 și 5 pentru care rezultatul este 20. În binar, folosind tehnica produselor parțiale, operația se derulează astfel :

00000100 x	- deînmulțitul
00000101	- înmulțitorul

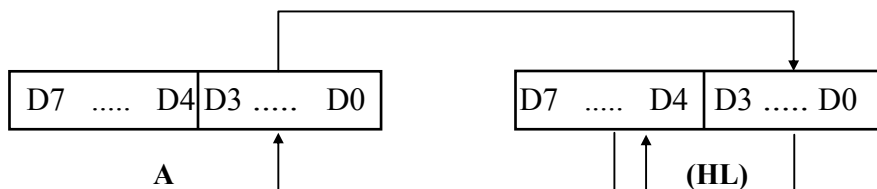
00000100	- primul produs parțial
00000000	- al doilea produs parțial
0000010000	- al treilea produs parțial

000001000100	- rezultatul $20_{(10)}$

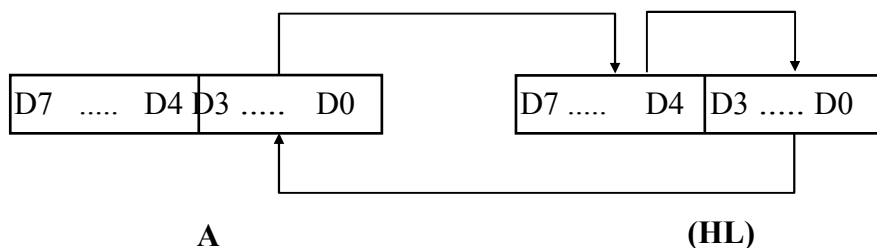
Se observă că produsele parțiale nenule apar pentru situațiile în care biții corespunzători din cadrul deînmulțitului sunt "1" logic. În aceste situații, produsul parțial reprezintă chiar deînmulțitul, deplasat la stânga cu acel număr de poziții, corespunzător poziției bitului din înmulțitor pentru care se efectuează respectivul produs parțial. Deplasarea se va putea realiza fie cu instrucțiuni de deplasare, fie cu instrucțiuni de rotație având grijă ca pe poziția bitului cmps să se aplice "0" logic.

C. Subgrupa operațiilor de rotație la nivel de nibble

a). **RLD** - (rotate left one digit) - rotație la nivel de nibble la stânga, între acumulator și o locație de memorie adresată de HL. Se face conform următoarei scheme, în care nibble - ul superior din A nu este afectat:



b). **RRD** - (rotate right one digit) - rotație la nivel de nibble la dreapta, între acumulator și o locație de memorie adresată de HL. Se face conform următoarei scheme, în care nibble-ul superior din A nu este afectat:



Îndrumar de laborator 5

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
RLD	* * 0 * 0 -	18	2	EDh 6Fh	A←85h HL←200h (200h)=01h RLD
RRD	* * 0 * 0 -	18	2	Deh 67h	A←80h (200)=18h A←85h HL←200h (200h)=01h RRD A←81h (200h)=05h

OBSERVAȚIE . Operațiile RLD și RRD pot fi folosite pentru despărțirea în nibble ai unui octet. Să considerăm în următorul exemplu $A = 00h$ și $(HL) = 29h$. După operația RRD , obținem $A = 09h$ și $(HL) = 02h$, adică cei doi nibble constituenți ai octetului inițial au fost despărțiți, formându-se doi noi octeți care au pe primii patru biți mai semnificativi “0” logic.

XV) . Grupa instrucțiunilor la nivel de bit

a). **BIT b , s** - testează bitul b din cadrul sursei s.

$s \in \{r, (HL), (IX+e), (IY+e)\}$; $b \in \{0,...,7\}$

Simbolic : $Z \leftarrow s_b$; bitul b al sursei s, negat trece în indicatorul de condiții Z .

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
BIT b ,r	- * 1 - 0 -	8	2	CBh 0 1 b b b s s s	B←81h BIT 7 ,B rezulta Z=0 BIT 0,B rezulta Z=1
BIT b ,(HL)	- * 1 - 0 -	12	2	CBh 0 1 b b b 1 1 0	
BIT b ,(IX + e)	- * 1 - 0 -	20	4	DDh, CBh ,e 0 1 b b b 1 1 0	
BIT b ,(IY +e)	- * 1 - 0 -	20	4	FDh, CBh, e 0 1 b b b 1 1 0	

b). **SET b , s** - setarea bitului b din sursa s

$s \in \{r, (HL), (IX+e), (IY+e)\}$; $b \in \{0,...,7\}$

Simbolic : $s_b \leftarrow 1$ Apar situațiile :

Îndrumar de laborator 5

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
SET b ,r	- - - - -	8	2	CBh 1 1 b b b s s s	HL←200h (200h)=00h SET 7,(HL) (200h)=80h
SET b ,(HL)	- - - - -	15	2	CBh 1 1 b b b 1 1 0	
SET b ,(IX + e)	- - - - -	23	4	DDh, CBh ,e 1 1 b b b 1 1 0	
SET b ,(IY + e)	- - - - -	23	4	FDh, CBh, e 1 1 b b b 1 1 0	

c). **RES b , s** - resetarea unui bit din sursa s

$s \in \{r, (HL), (IX+e), (IY+e)\}$; $b \in \{0,...,7\}$

Simbolic : $s_b \leftarrow 0$

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
RES b ,r	- - - - -	8	2	CBh 1 0 b b b s s s	HL←200h (200h)=80h RES 7,(HL) (200h)=00h
RES b ,(HL)	- - - - -	15	2	CBh 1 0 b b b 1 1 0	
RES b ,(IX + e)	- - - - -	23	4	DDh, CBh ,e 1 0 b b b 1 1 0	
RES b ,(IY + e)	- - - - -	23	4	FDh, CBh, e 1 0 b b b 1 1 0	

XVI) . Grupa instrucțiunilor de intrare - ieșire

Prin intermediul instrucțiunilor din această grupă, procesorul poate să interacționeze cu circuitele I/O. În cadrul execuției acestor instrucțiuni, apar cicli de tip citire I/O, respectiv, scriere I/O, cicli în care semnalul IORQ se activează.

A. Subgrupa instrucțiunilor de citire a porturilor de intrare

a). **IN A , (n8)** - transferă în A conținutul portului de intrare cu adresa specificată prin constanta pe 8 biți, n8.

Simbolic : $A \leftarrow (n8)$

b). **IN r , (C)** - transferă în registrul general r, conținutul portului de intrare cu adresa pe 8 biți, precizată în registrul C.

Simbolic : $r \leftarrow (C)$

c). **INI** - (input & increment) - citește portul de intrare și incrementează

Simbolic :

(HL)	$\Leftarrow$	(C)
HL	$\Leftarrow$	HL+1
B	$\Leftarrow$	B - 1

Instrucțiunea depune în memorie, la adresa specificată de registrul dublu HL, conținutul portului de intrare adresat prin intermediul conținutului registrului C. Adresa de memorie (conținutul registrului dublu HL) se incrementează la execuția instrucțiunii, în vederea pregătirii unei depuneri ulterioare, prin aceeași instrucțiune. Registrul B poate fi folosit pe post de contor al numărului de citiri ce se vor efectua, el decrementându-se la fiecare citire și depunere.

Pentru ca instrucțiunea să se deruleze corect este necesar ca în prealabil să încarcăm:

- în registrul C, adresa pe 8 biți a portului de intrare;
- în registrul HL, adresa de început a zonei de memorie în care se va face depunerea;
- în registrul B, numărul de citiri și depuneri ce se vor efectua, dacă instrucțiunea va fi rulată repetat.

d). **INIR** - (input, increment & repeat) - citește, incrementează și repetă

Simbolic :

(HL)	$\Leftarrow$	(C)
HL	$\Leftarrow$	HL + 1
B	$\Leftarrow$	B - 1

- se repetă până când B = 0

Sunt valabile toate observațiile prezentate la instrucțiunea anterioară, cu precizarea că se execută repetat până când B = 0. Numărul maxim de repetări este dat de capacitatea maximă a lui B, adică 255.

e). **IND** - (input & decrement) - citește și decrementează

Simbolic :

(HL)	$\Leftarrow$	(C)
HL	$\Leftarrow$	HL - 1
B	$\Leftarrow$	B - 1

Sunt valabile observațiile de la instrucțiunea INI, cu precizarea că în HL, inițial, trebuie să se încarce adresa de sfârșit a zonei în care se va face depunerea; la execuție repetată a instrucțiunii, transferurile se fac de la sfârșitul zonei de memorie către începutul acesteia.

Îndrumar de laborator 5

f). **INDR** - (input, decrement & repeat) - citește, decrementează și repetă

Simbolic :

$$\begin{aligned} (\text{HL}) &\leftarrow (\text{C}) \\ \text{HL} &\leftarrow \text{HL} - 1 \\ \text{B} &\leftarrow \text{B} - 1 \end{aligned} \quad \text{- se repetă până când B=0}$$

Această instrucțiune este practic instrucțiunea anterioară IND, executată repetat de numărul indicat de registrul B.

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
IN A ,(n8)	- - - - -	11	2	DBh n8	(29h)=44h IN A ,(29h) A←44h
IN r ,(C)	* * * * 0 -	12	2	EDh 01 d d d 0 0 0	A←44h
INI	- * - - 1 -	16	2	EDh A2h	HL←200h
INIR	- 1 - - 1 -	16	2	EDh B2h	C←20h
IND	- * - - 1 -	16	2	EDh AAh	B←10h (20h)=55h IND
INDR	- 1 - - 1 -	16	2	EDh BAh	(200h)=55h B←0Fh

B. Subgrupa instrucțiunilor de scriere a porturilor de ieșire

a). **OUT (n8) , A** - transferă conținutul acumulatorului la portul de ieșire cu adresa specificată de constanta pe 8 biți, n8.

Simbolic : $(\text{n8}) \leftarrow \text{A}$

b). **OUT (C) , r** - transferă conținutul acumulatorului la portul de ieșire cu adresa specificată în registrul C; în prealabil, în registrul C trebuie încărcată adresa portului de ieșire în care se vor depune datele.

Simbolic : $(\text{C}) \leftarrow \text{r}$

c). **OUTI** - (output & increment) - ieșire și incrementare ;

Simbolic:

$$\begin{aligned} (\text{C}) &\leftarrow (\text{HL}) \\ \text{HL} &\leftarrow \text{HL} + 1 \\ \text{B} &\leftarrow \text{B} - 1 \end{aligned}$$

Îndrumar de laborator 5

Pentru ca această instrucțiune să se deruleze corect este necesar să încărcăm:

- în registrul C, adresa portului de ieșire cu care se va lucra ;
- în registrul dublu HL, adresa de început a zonei de memorie de la care se va face preluarea în vederea transferului ;
- în registrul B, numărul de preluări și scrieri ce se vor efectua , dacă instrucțiunea se va efectua repetat.

Instrucțiunea se autopregătește pentru o nouă rulare, prin incrementarea lui HL și decrementarea lui B.

d). **OTIR** - (output, increment & repeat) - ieșire , incrementare și repetă

Simbolic :

$$\begin{array}{lll} (C) & \Leftarrow & (HL) \\ HL & \Leftarrow & HL + 1 \\ B & \Leftarrow & B - 1 \end{array} \quad \text{- se repetă până când } B=0$$

Este practic instrucțiunea anterioară (INI), dar executată repetat de numărul de ori indicat de registrul B.

e). **OUTD** - (output & decrement) - ieșire și decrementare

Simbolic:

$$\begin{array}{lll} (C) & \Leftarrow & (HL) \\ HL & \Leftarrow & HL - 1 \\ B & \Leftarrow & B - 1 \end{array}$$

Sunt valabile precizările de la instrucțiunea OUTI, cu observația că în registrul HL, în prealabil, trebuie încărcată adresa de sfârșit a zonei de memorie de la care se fac preluările. La execuția repetată a instrucțiunii, preluările din memorie se fac de la adresa de sfârșit către adresa de început a zonei.

f). **OTDR** - (output, decrement & repeat) - ieșire , decrementare și repetare

Simbolic:

$$\begin{array}{lll} (C) & \Leftarrow & (HL) \\ HL & \Leftarrow & HL - 1 \\ B & \Leftarrow & B - 1 \end{array} \quad \text{- se repetă până când } B=0$$

Sunt valabile precizările făcute la instrucțiunea anterioară, cu observația ca instrucțiunea se executa repetat de numărul de ori indicat de registrul B.

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
OUT (n8) ,A	-	-	-	-	-	-	11	2	D3h n8	A←44h OUT A ,(33h)
									EDh	(33h)=44h
OUT (C) ,r	-	-	-	-	-	-	12	2		

Îndrumar de laborator 5

				0 1 s s s 0 0 1	HL←200h C←20h B←10h (200h)=35h OUTI (20h)=35h B←0Fh
OUTI	- * - - 1 -	16	2	EDh A3h	
OTIR	- 1 - - 1 -	16	2	EDh B3h	
OUTD	- * - - 1 -	16	2	EDh ABh	
OTDR	- 1 - - 1 -	16	2	EDh BBh	

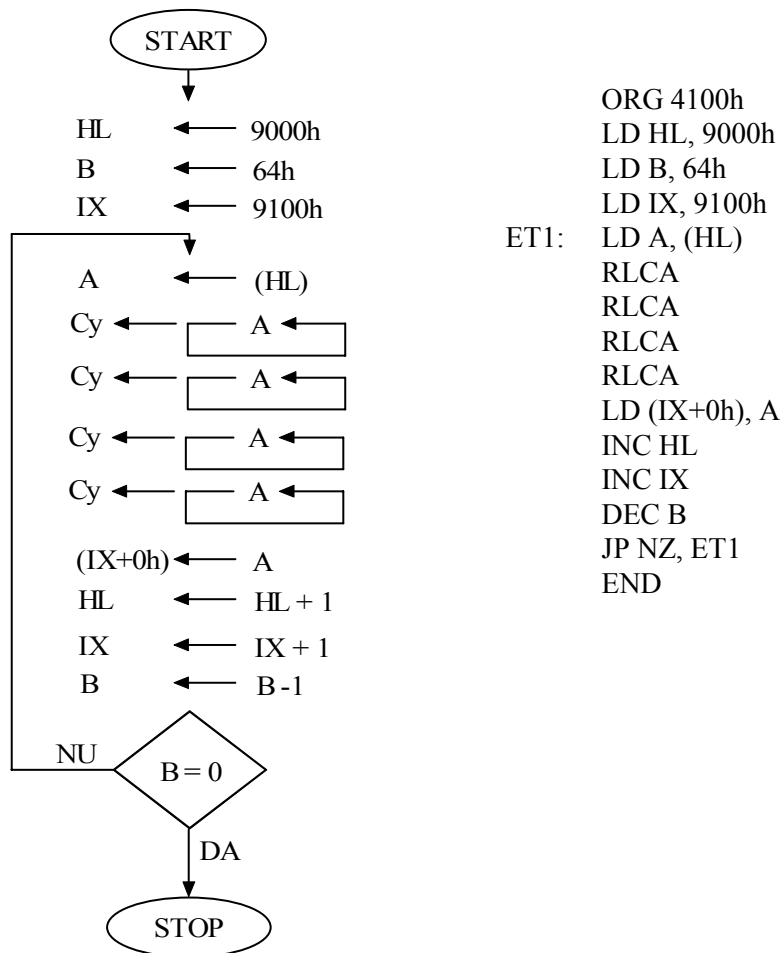
3. PROBLEME REZOLVATE

1 Se consideră un șir de numere aflat în memoria unui microsistem cu microprocesor Z-80, începând cu adresa 9000h. Mărimea șirului este de 100 numere, reprezentate pe un octet. Să se codeze acest șir prin schimbarea nibble-ului superior cu nibble-ul inferior din fiecare octet și plasarea noului șir începând cu adresa 9100h.

Soluție:

Operație de codare constă în efectuarea a patru rotații la stânga (sau la dreapta) a biților fiecărui octet. Rezolvarea este prezentată în continuare:

Îndrumar de laborator 5



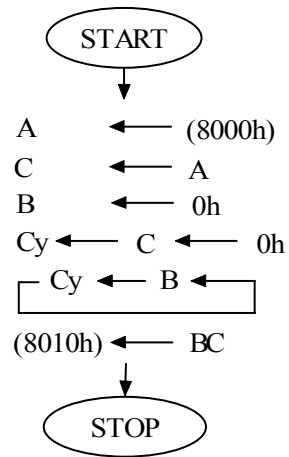
2. În memoria unui microsistem cu microprocesor Z-80 se găsește la adresa 8000h un număr reprezentat pe un octet. Să se efectueze asupra sa următoarele operații :

- să se înmulțească cu 2 și să se plaseze rezultatul începând cu adresa 8010h
 - să se înmulțească cu 8 și să se plaseze rezultatul începând cu adresa 8020h
 - să se împartă la 2 și să se plaseze câtul la adresa 8030h și restul la adresa 8031h
 - să se împartă la 8 și să se plaseze câtul la adresa 8040h și restul la adresa 8041h
- Operațiile vor fi efectuate prin instrucțiuni de deplasare.

Soluție:

a)

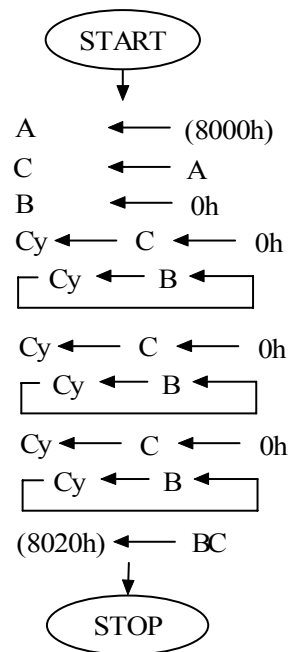
Îndrumar de laborator 5



```

ORG 4000h
LD A, (8000h)
LD C, A
LD B, 0h
SLA C
RL B
LD (8010h), BC
END
  
```

b)

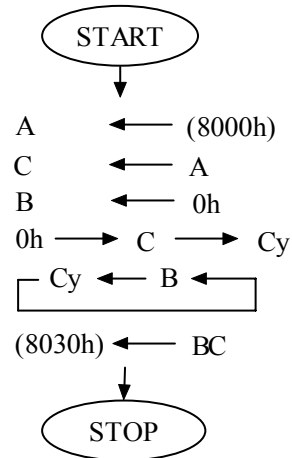


```

ORG 4000h
LD A, (8000h)
LD C, A
LD B, 0h
SLA C
RL B
SLA C
RL B
SLA C
RL B
LD (8020h), BC
END
  
```

Îndrumar de laborator 5

c)

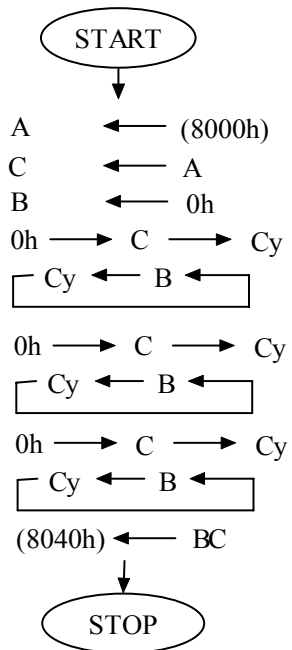


```

ORG 4000h
LD A, (8000h)
LD C, A
LD B, 0h
SRL C
RL B
LD (8030h), BC
END

```

d)



```

ORG 4000h
LD A, (8000h)
LD C, A
LD B, 0h
SRL C
RL B
SRL C
RL B
SRL C
RL B
LD (8040h), BC
END

```

3. Să se realizeze un program care să convertească în cod ascii orice număr hexazecimal exprimat pe un octet. Inițial numărul se află în memorie la locația cu adresa 8200h. Rezultatul va fi depus începând cu adresa 8300h.

Soluție

Practic, problema cere ca plecând de la octetul exprimat în binar (hexazecimal), să obținem doi octeți care reprezintă codurile ASCII ale celor două "jumătăți" (nibble) ale octetului inițial. În cod ASCII cifrele, de la 0 până la 9, au codurile ASCII cuprinse între 30h și 39h, respectiv. Pentru literele mari (majusculile) codurile ASCII încep de la 41h (pentru "A"), continuând cu păstrarea ordinii alfabetic. Ținând seama de aceste particularități ale codului ASCII, este nevoie de o corecție care trebuie făcută în felul următor :

- dacă numărul binar (hexazecimal) este cuprins între 00h și 09h, corecția se face prin adunare cu 30h.

De exemplu : 03h are în cod ASCII echivalentul 33h

corecția implică: $03h + 30h = 33h$

- dacă numărul binar (hexazecimal) este cuprins între 0Ah și 0Fh, corecția se face prin adunare cu 37h.

De exemplu : 0Bh are în cod ASCII echivalentul 42h

corecția implică: $0Bh + 37h = 42h$

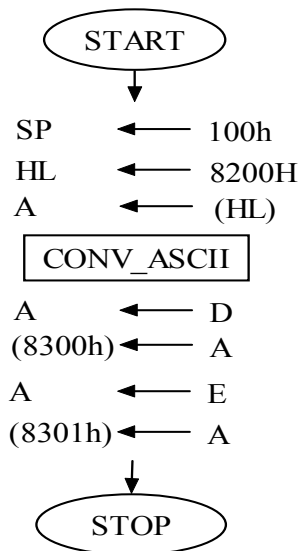
Pentru rezolvarea problemei cerute rezultă că va trebui să "spargem" octetul de convertit în două părți, obținându-se astfel doi octeți în care cei doi nibble ai octetului inițial să apară pe pozițiile cmpps ale acestora. Cei 4 biți mai semnificativi ai acestor doi octeți trebuie să fie 0. Va urma testul de apartenență în una din cele două categorii anterioare (cifra sau litera) și realizarea corecției corespunzătoare.

Spre exemplu, pentru numărul binar $10110011_{(2)}$, va trebui afișat în hexazecimal B3h. În acest sens va trebui să obținem doi octeți 42h (codul ASCII pentru B) și 33h (codul ASCII pentru 3).

Organizăm programul sub forma unui program principal ce apelează o subrutină care realizează conversia ASCII.

Inițial, subrutina ce realizează conversia (CONV_ASCII) primește în acumulator octetul de convertit. Subrutina folosește locația de memorie cu adresa 8201h, pentru execuția instrucțiunii RRD, în vederea despărțirii în nibble ai octetului. După operația RRD, în acumulator se obține ocmpps-ul din viitorul echivalent ASCII, iar în locația de memorie adresată de registrul HL, ocms-ul. Urmează corecția ASCII pentru rezultatul din acumulator și plasarea acestuia în registrul E, respectiv, corecția conținutului locației de memorie adresată de HL și plasarea rezultatului obținut în registrul D. În concluzie, subrutina de conversie ASCII primește în A octetul de convertit și plasează rezultatul (2 octeți) în regiștrii D și E.

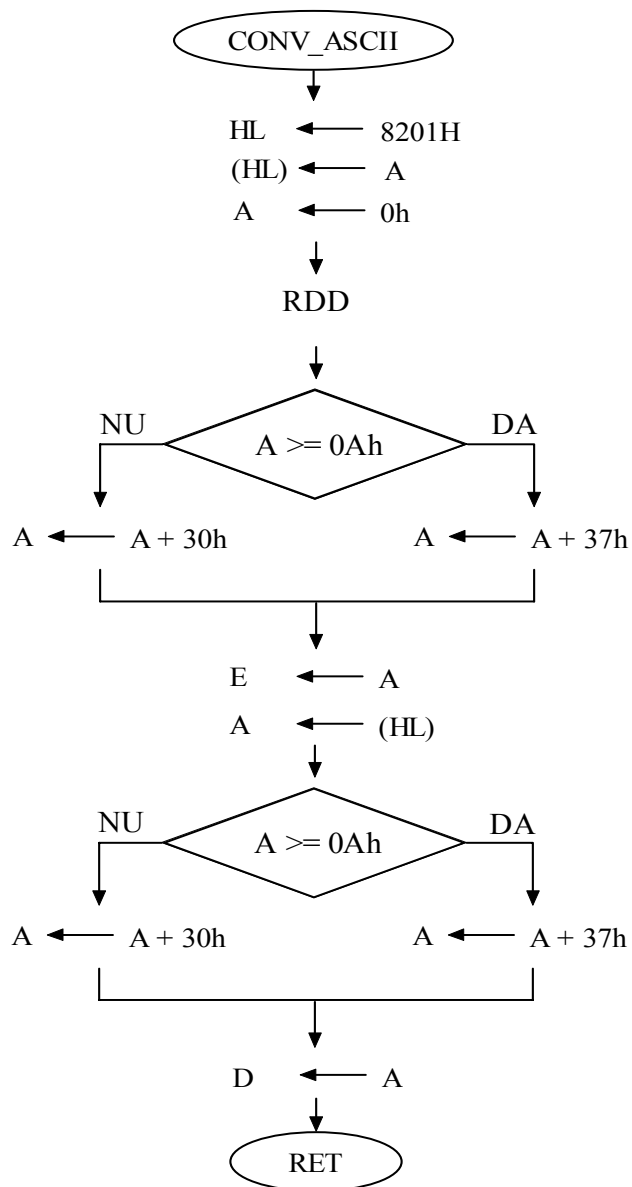
program principal



ORG 100h	
LD SP,100h	100h: 31h 00h 01h
LD HL,200h	103h: 21h 00h 02h
LD A, (HL)	106h: 7Eh
CALL CONV_ASCII	107h: CDh 00h 03h
LD A, D	10Ah: 7Ah
LD (8300h), A	10Bh: 32h 00h 83h
LD A, E	10Eh: 7Bh
LD (8301h), A	10Fh: 32h 01h 83h
HALT	112h: 76h

Îndrumar de laborator 5

subrutina de conversie în cod ascii



```

ORG 300h
CONV_ASCII:
    LD HL,201h
    LD (HL),A
    LD A,00h
    RRD
    CP 0Ah
    JP Z,ET1
    JP C,ET2
ET1:  ADD A,37h
    JP,ET3
ET2:  ADD A,30h
ET3:  LD E,A
    LD A,(HL)
    CP 0Ah
    JP Z,ET4
    JP C,ET5
ET4:  ADD A,37h
    JP ET6
ET5:  ADD A,30h
ET6:  LD D,A
    RET
  
```

Îndrumar de laborator 5

4. Se consideră două numere binare reprezentate pe câte un octet fiecare. Primul număr este plasat în memorie la adresa 8300h, iar cel de-al doilea număr la adresa 8400h. Se cere realizarea unui program pentru efectuarea operației de înmulțire între cele două numere cu plasarea rezultatului în memorie de la adresa 8500h, începând cu ocmpps-ul.

Soluție:

Un algoritm rapid de efectuare a operației de înmulțire presupune obținerea de produse parțiale și cumularea lor imediat după obținere într-un produs parțial. Această metodă a fost explicată pe larg la prezentarea instrucțiunii SRL (vezi mai sus).

Pentru situația înmulțirii a 2 octeți sunt necesare următoarele locații:

- 2 octeți pentru cumularea produselor parțiale și obținerea rezultatului final. Vom folosi pentru aceasta registrul dublu HL.

OBS: - registrul HL permite operații de genul:

$HL \leftarrow HL + DE$ sau $HL \leftarrow HL + BC$

- la inițializare $HL \leftarrow 0h$

- pentru obținerea produselor parțiale vom folosi registrul dublu DE

OBS: - la inițializare $D \leftarrow 0h$ și $E \leftarrow deînmu\lftit$

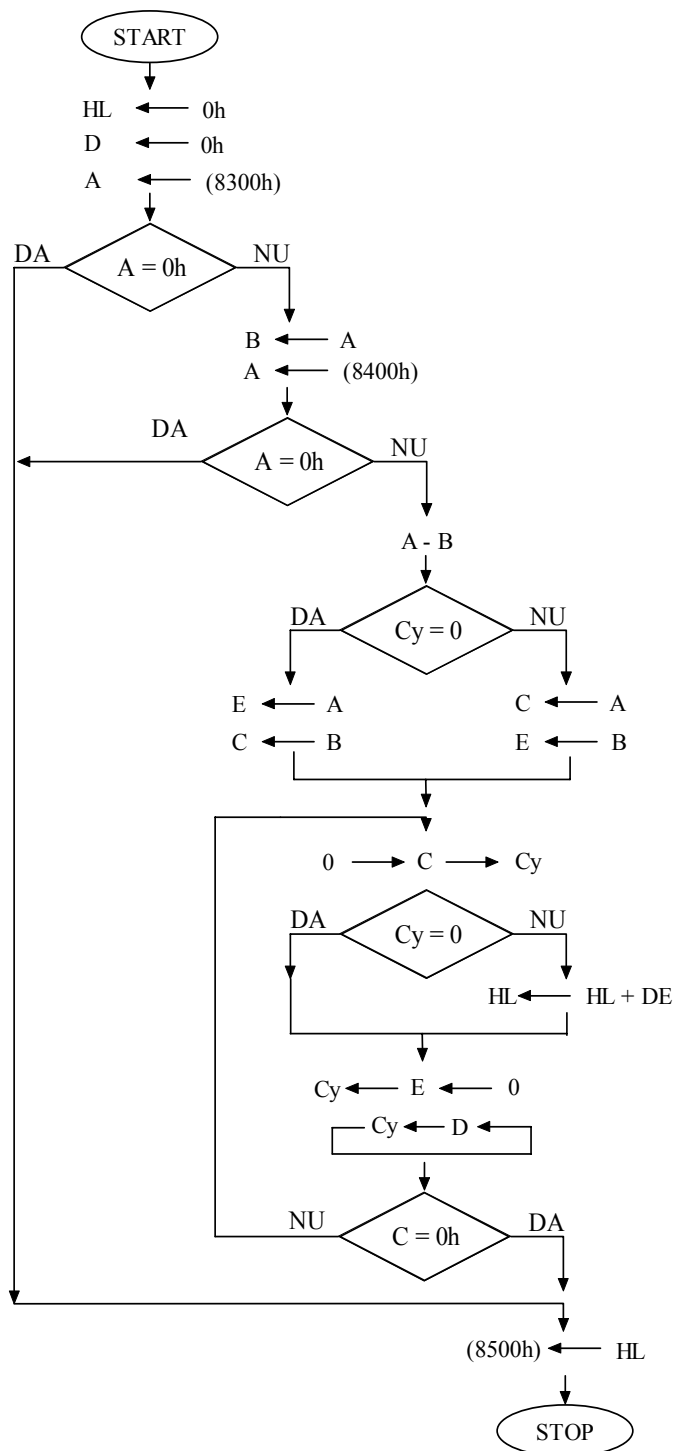
- vom păstra înmulțitorul în registrul C. Asupra acestuia la fiecare pas se va efectua o operație de shift-are la dreapta. Algoritmul se va încheia în momentul în care C devine 0h

Se va avea în vedere situația în care unul din cei doi operanzi e zero, caz în care rezultatul e 0 și nu mai e necesară efectuarea operației de înmulțire.

O altă măsură de optimizare constă în determinarea celui mai mare dintre operanzi și considerarea lui ca deînmulțit. În acest fel se reduce numărul de pași pe care algoritmul îl face.

* programul sursă *

Îndrumar de laborator 5



```

ORG 4000h
LD HL, 0h
LD D, 0h
LD A, (8300h)
CP 0h
JP Z, ET1
LD B, A
LD A, (8400h)
CP 0h
JP Z, ET1
CP B
JP NC, ET2
LD E, A
LD C, B
JP ET3
ET2: LD C, A
      LD E, B
ET3: SRL C
      JP NC, ET4
      ADD HL, DE
ET4: SLA E
      RL D
      LD A, C
      CP 0h
      JP NZ, ET3
ET1: LD (8500h), HL
      END
  
```

Îndrumar de laborator 5

5. Se consideră un număr binar reprezentat pe un octet și plasat în memoria unui microsistem la adresa 8500h. Se cere realizarea unui program pentru obținerea echivalentului său BCD și plasarea acestuia în memorie de la adresa 8600h începând cu ocmps. Rezolvarea se va efectua pe caz general.

Soluție:

Se consideră numărul binar : $11010111_{(2)} = D7h$

Formula de calcul a valorii zecimale este:

$$2^7 \cdot 1 + 2^6 \cdot 1 + 2^5 \cdot 0 + 2^4 \cdot 1 + 2^3 \cdot 0 + 2^2 \cdot 1 + 2^1 \cdot 1 + 2^0 \cdot 1 = 215$$

Pe caz general avem : $X_n X_{n-1} X_{n-2} X_{n-3} \dots X_1 X_0$

În formula de mai sus prin scoaterea de factor comun pe 2 în mod repetat se ajunge la formula:

$$\text{Valoarea zecimală} = 2(\dots 2(2X_n + X_{n-1}) + X_{n-2}) + X_{n-3} \dots) + X_0$$

La fiecare pas se dublează numărul precedent și se adună bitul următor.

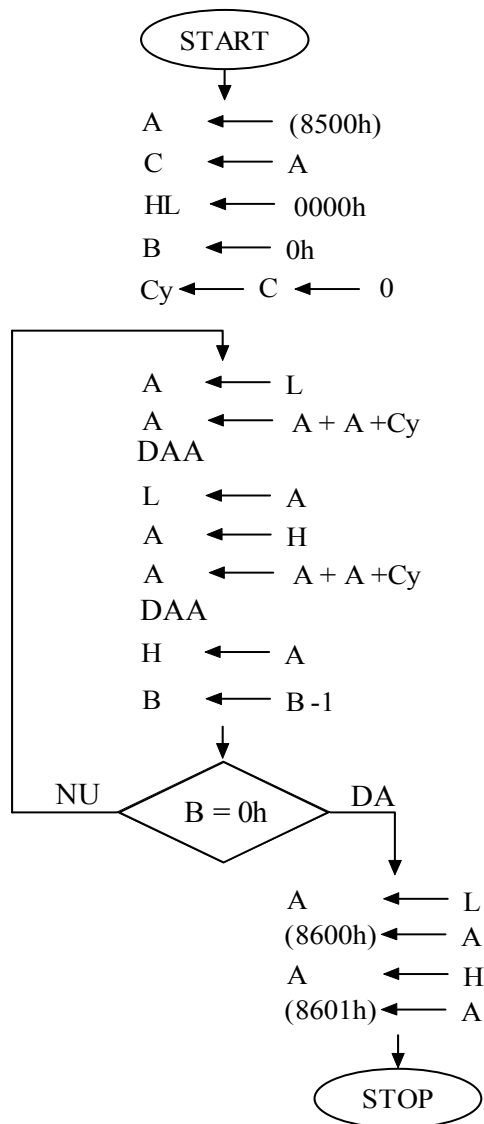
După n pași se ajunge la numărul zecimal căutat.

La dublarea numărului trebuie avut în vedere ca asupra lui trebuie aplicată o corecție zecimală. Aceasta se face prin instrucțiunea DAA. Reamintim că această instrucțiune este valabilă operația a fost una aritmetică și numărul supus operației a fost exprimat în BCD.

În rezolvarea problemei s-au folosit următoarele locații:

- registrul C pentru păstrarea numărului binar. El va fi deplasat la stânga cu o poziție la fiecare pas în vederea obținerii numărului biților numărului de convertit
- registrul B pentru menținerea numărului de biți al numărului binar de convertit (în acest caz $B = 8$)
- rezultatul unei astfel de conversii se reprezintă pe 2 octeți și pentru aceasta s-a folosit registrul HL. Initial acesta este inițializat cu 0h

Organigrama și listingul programului sunt prezentate mai jos:



```

ORG 4000h
LD A, (8500h)
LD C, A
LD HL, 0h
LD B, 0h
ET1: SLA C
LD A, L
ADC A, A
DAA
LD L, A
LD A, H
ADC A, A
DAA
LD H, A
DEC B
JP NZ, ET1
LD A, L
LD (8600h), A
LD A, H
LD (8601h), A
END
    
```

4. DESFĂȘURAREA LUCRĂRII

1. Se vor citi și prelua în scris toate informațiile teoretice prezentate până în acest moment. Se atrage atenția că aceste informații vor fi folosite și la desfășurarea altor lucrări de laborator.
2. Se vor studia problemele rezolvate, încercând găsirea altor posibilități de rezolvare.

Îndrumar de laborator 5

3. Programele obiect ale problemelor rezolvate vor fi introduse în memoria microsistemelor existente în laborator (prin intermediul programului monitor) și se vor rula. Se va observa efectul rulării și buna lor execuție.

4. Se vor rezolva următoarele probleme propuse și se va urmări execuția corectă prin introducerea programelor obiect în memoria microsistemelor și rulare:

a). Se consideră două numere reprezentate pe câte 6 octeți fiecare. Primul număr este plasat în memoria unui microsistem cu microprocesor Z-80, începând cu adresa 1000h și cu octetul cel mai puțin semnificativ, iar al doilea număr de la adresa 2000h, tot începând cu octetul cel mai puțin semnificativ. Se cere realizarea operației de adunare între cele două numere cu plasarea rezultatului de la adresa 3000h începând cu octetul cel mai puțin semnificativ. Adunarea se va trata pe caz general, iar algoritmul se dorește cât mai optim cu putință.

b). Se consideră două numere binare reprezentate pe câte un octet fiecare. Primul număr este plasat în memorie la adresa 300h, iar cel de-al doilea număr la adresa 400h. Se cere realizarea unui program pentru efectuarea operației de înmulțire între cele două numere cu plasarea rezultatului în memorie de la adresa 500h, începând cu ocmps-ul.

c). Se consideră două numere reprezentate pe câte doi octeți fiecare. Primul număr este plasat în memoria unui microsistem de la adresa 200h, începând cu ocmps-ul, iar cel de-al doilea număr se află la adresa 300h, începând tot cu ocmps-ul. Se cere realizarea operației de înmulțire între cele două numere (tratare pe caz general) cu plasarea rezultatului începând cu adresa 400h și cu ocmps-ul.

d). Se consideră un număr binar reprezentat pe un octet și plasat în memoria unui microsistem la adresa 500h. Se cere realizarea unui program pentru obținerea echivalentului sau BCD și plasarea acestuia în memorie de la adresa 600h începând cu ocmps. Rezolvarea se va efectua pe caz general.

e). Se consideră un număr BCD mai mic decât $255_{(10)}$. Acest număr este plasat în memoria unui microsistem de la adresa 700h, începând cu ocmps. Se cere realizarea unui program pentru obținerea echivalentului său binar și plasarea acestuia în memorie de la adresa 800h.

Îndrumar de laborator 5